PetClinic DevOps Capstone

Project Documentation — Team 4

August 2026

This document brings together each team member's part of the project — infrastructure, containerization, deployment automation, and CI/CD — in their own words, followed by a full walkthrough of how it all fits together as one pipeline.

Team

Name	Area
Parwez Akhtar	Mentor
Talha Keles	Team Lead — Infrastructure, Terraform & Security
Sandis Sarkovskis	Docker & Docker Hub
Arina Petrova	Ansible, NGINX & Blue/Green Deployment
Reinis Janis Brencis	Jenkins & CI/CD

Infrastructure & Security

Talha Keles

I set up the AWS infrastructure for the project using Terraform, and later handled a security requirement that came up mid-project — migrating from SSH to AWS Systems Manager.

Terraform (infra/)

Everything runs on a single EC2 instance (Amazon Linux 2023, t3.medium), provisioned entirely through Terraform:

- main.tf — the EC2 instance itself. Uses a data lookup for the latest Amazon Linux 2023 AMI instead of a hardcoded ID, so it doesn't go stale over time.
- security_group.tf — firewall rules. Only ports 80 (NGINX) and 8081 (Jenkins) are open to the internet. Port 8080 used to be open too, a leftover from before blue/green, and was removed once nothing was listening on it anymore. SSH is not in there at all.
- key_pair.tf — SSH key pair, originally used for access, now mostly a leftover from before the SSM migration.
- install.sh — runs once automatically when the instance first boots. Installs and starts Docker.
- cloudwatch.tf — a CPU alarm (triggers above 80% for two consecutive checks) and a log group for application logs.
- eip.tf — attaches an Elastic IP so the public IP doesn't change if the instance restarts.
- outputs.tf — prints the instance ID and public IP after every apply.
- destroy.sh — tears everything down, but requires typing a confirmation phrase first so nobody destroys the instance by accident.

Remote Terraform backend (backend-setup/)

Terraform's state file — which tracks what infrastructure exists — now lives in an S3 bucket instead of on my laptop, with a DynamoDB table used to lock it so two people can't run terraform apply at the same time and corrupt it. This is a separate, small Terraform project of its own, since the bucket that stores the state can't be created by the same project it's storing state for.

SSH → SSM migration

Partway through the project, Accenture's Security team required that no team use SSH keys or open SSH ports — everything had to go through AWS Systems Manager (SSM) instead. This was reinforced by another team's instance actually getting terminated for being flagged as a DoS source.

What changed:

- Removed the SSH ingress rule from the security group entirely.
- Attached an IAM role to the instance that allows SSM Session Manager to connect.
- Everyone on the team switched from SSH with a shared key file to “aws ssm start-session --target <instance-id>”, which authenticates through each person's own AWS login (with MFA) instead.

This also meant retiring the shared private key that had been floating around in Discord DMs, which in hindsight wasn't a great practice to begin with. Later I also stopped and disabled the sshd service directly on the instance, not just blocked it at the security group level — the security group already made SSH unreachable, but shutting off the service itself felt like the more thorough thing to do.

Permissions

The bootcamp AWS account has a policy that blocks everyone from creating new IAM roles — intentional on the organizers' side, not a bug. It affected getting CloudWatch logging working and the DynamoDB table for the remote backend, and had to be resolved by asking the organizers to either grant a specific permission or set up a role for us, rather than working around it ourselves.

Bonus tasks

- Remote Terraform backend (S3 + DynamoDB)
- CloudWatch logging & CPU alarm
- Automated, confirmation-guarded terraform destroy script

What I'd improve with more time

- Add an email/SNS notification on the CloudWatch alarm, so it actually alerts someone
- Automate the MFA session refresh instead of doing it manually every few hours
- Clean up the now-unused SSH key pair resource from the Terraform code

AI Usage

Used Claude (Sonnet 5, Anthropic) mostly for troubleshooting and error fixes. Used it for planning phase of the project. Also while creating terraform files I’ve asked how can I handle this & implement this then it gave me some options then I’ve implemented some of it inside it. Because of I am not experienced that much I had some problems with permission problems, so whenever I have a permission error I asked it what kind of permission I need from Accenture side then I’ve asked for that permissions from Karina and Rizwan. To be honest I was seeing which permission I need from the error output but I wanted to be sure before asking those from them. Also while creating README file and documentation, I took documentation files from my teammates which shows everything from their side and asked AI to make it clear and style it. Also asked AI what kind of demo we should do to show all work, so it gave some recommendations and I applied step by step. While creating the architecture diagram I’ve explained our project then it gave some options but it wasn’t that good and correct so I’ve changed it but used that as a base. Also I’ve asked what kind of practice is usual for cloudwatch settings like % of cpu usage. Also used it for learning specific terminal commands.

Docker & Docker Hub

Sandis Sarkovskis

My part of the project was containerizing the PetClinic application and getting the image published to Docker Hub, plus the bonus task of versioning images with the Jenkins build number.

Writing the Dockerfile

The Dockerfile uses a two-stage build. The first stage uses a Maven image with a full JDK to compile the app into a jar. The second stage starts fresh from a small Alpine image that only has the Java runtime, and just copies the finished jar over — Maven and the JDK never ship in the final image, which keeps it a lot smaller.

One detail worth explaining: pom.xml is copied and dependencies are downloaded before the source code is copied in. Docker caches each step, so as long as dependencies haven't changed, rebuilding after a code change skips the download step entirely and goes straight to compiling — makes repeat builds much faster.

A health check pings the homepage every 15 seconds, with a 40-second grace period before failures count, since a Spring Boot app genuinely takes a while to start up. It uses wget rather than curl because the Alpine base image doesn't ship with curl.

After the work was verified, the branch was merged into main through a pull request (PR #18).

Building and testing it locally

Before pushing anything, the image was built and run on the EC2 instance to confirm it actually works:

```
docker build -t sandissarkovskis/devops-final-project:test .

docker run -d -p 9090:8080 --name petclinic-test sandissarkovskis/devops-final-project:test

curl -I http://localhost:9090/
```

The curl response came back HTTP/1.1 200, confirming the app was up and serving the PetClinic homepage from inside the container. Port 9090 was used on the host side since Jenkins was already occupying 8080 on that machine — inside the container the app still runs on 8080 as normal.

Publishing to Docker Hub

A public repository was created on Docker Hub: sandissarkovskis/devops-final-project. For authentication, a personal access token was generated in Docker Hub's security settings instead of using the account password — the recommended approach, since a token can be revoked at any time without touching the account itself.

Since the repository is public, pulling the image doesn't require any login at all — the token is only needed for pushing. That means Jenkins, or anyone on the team, can pull the image without needing Sandis's credentials.

Bonus task: versioned images with the Jenkins build number

Only ever pushing a “latest” tag has a real downside: there's no way to tell exactly which build is running, and nothing older to roll back to. So every pipeline run tags the image with Jenkins' build number as well as latest:

```
docker build -t sandissarkovskis/devops-final-project:${BUILD_NUMBER} \
  -t sandissarkovskis/devops-final-project:latest .

docker push sandissarkovskis/devops-final-project:${BUILD_NUMBER}

docker push sandissarkovskis/devops-final-project:latest
```

BUILD_NUMBER is a variable Jenkins provides automatically — it goes up by one with every run, so each image ends up with its own unique tag. The latest tag still exists as a convenient pointer to the newest one, but the numbered tags are what

make the history traceable: you can see exactly which build produced which image on Docker Hub, and rolling back is just a matter of deploying an older number.

AI Usage

I used AI for writing the dockerfile, basically when I got stuck or something wasn’t working I used it as well, everyone used it for the documentation.

Ansible Deployment & Blue/Green

Arina Petrova

This part covers deployment automation on the EC2 instance: configuring NGINX as a reverse proxy, implementing a blue/green deployment process, and preparing everything for integration with the Jenkins CI/CD pipeline.

Purpose

Instead of manually connecting to the server and running Docker, package installation, and service-management commands by hand, the required deployment steps are described in an Ansible playbook. That makes deployment repeatable and consistent — the same playbook can be run again after application changes or on a recreated environment — and it gives Jenkins a clear integration point to trigger deployment automatically.

Project structure and execution model

The Ansible configuration lives under the ansible/ directory: inventory.ini, playbook.yml, and templates/petclinic.conf.j2 (a Jinja2 template used to generate the NGINX config).

The inventory defines an app group with localhost and ansible_connection=local — Ansible runs directly on the EC2 instance rather than connecting out to a separate target. When the team's access method changed from SSH to AWS Systems Manager Session Manager, the Ansible inventory didn't need to change at all, because how a person gets into the instance is separate from how the playbook itself executes. Ansible just runs locally on the box, regardless of how someone connected to it.

Docker deployment automation

The first version of the playbook automated a standard single-container deployment: ensuring Docker was running, pulling the PetClinic image, removing the previous container if needed, and starting the new one. Docker operations are handled with the community.docker Ansible collection, so resources are managed declaratively instead of through a sequence of manual docker commands.

NGINX reverse proxy (bonus)

NGINX was added in front of the application. Before this, the app was reached directly on its own port; with the reverse proxy, external requests come in through port 80 and NGINX forwards them to the active container.

NGINX installation is automated through the system package manager, and the playbook ensures the service is started and enabled so it survives a reboot. The config is generated from the petclinic.conf.j2 template and deployed to /etc/nginx/conf.d/petclinic.conf, including standard forwarding headers (Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto). Before it's applied, the playbook runs “nginx -t” to validate the config and avoid deploying something broken.

A handler named “Restart NGINX” only restarts the service when the generated configuration actually changes, avoiding unnecessary restarts. During the blue/green switch, flush_handlers is used at the right point so a pending restart happens immediately, before the old deployment is removed.

Blue/green deployment design

Two deployment slots are defined: petclinic-blue on host port 8082 and petclinic-green on host port 8083 (8081 was already taken by Jenkins). The NGINX template no longer points at a fixed backend port — it uses an active_port variable, so the same template can generate a config pointing at either slot.

Ports 8082 and 8083 don't need to be opened in the EC2 security group. They're backend ports used only locally on the instance; users reach the app through port 80, and NGINX forwards internally to whichever container is currently active.

Health checks and the switch

Ansible pulls the image and starts the Blue container, then runs an HTTP health check (via the Ansible URI module, expecting status 200), with retries and a delay since Spring Boot needs time to start. The deployment doesn't continue to traffic-switching until that check passes — if the app doesn't come up, the playbook fails rather than pointing NGINX at something broken.

The same thing then happens for Green. Once Green returns HTTP 200, active_port is switched to Green, NGINX config is regenerated and validated, the service is confirmed running, and handlers are flushed so the switch takes effect immediately. Only after that successful switch is the old Blue container removed — so the previous working version stays available the entire time the new one is being verified.

Dynamic image tags for Jenkins

The playbook defines an image_tag variable (default: latest, so a manual deployment is still possible) and builds the full image name from it. Jenkins overrides this at runtime with its BUILD_NUMBER — so if Jenkins build 42 produces image :42, Jenkins passes image_tag=42 and the playbook deploys exactly that image. This is what connects the CI and CD sides: Jenkins builds and pushes the versioned image, Ansible deploys and validates it.

Testing

Testing was incremental: connectivity first (Ansible ping), then playbook --syntax-check, then the original single-container deployment (verified with failed=0 and a healthy container), then NGINX installation and config validation, and finally the full blue/green flow — both Blue and Green passing health checks, NGINX config confirmed to point at localhost:8083, the old Blue container removed, and the app reachable through NGINX on port 80.

Bonus tasks

- NGINX reverse proxy
- Blue/green deployment with health checks and zero-downtime switching
- Support for Jenkins-provided versioned image tags

AI Usage

As cannot give specific prompts, I just have few big dialogs with chatgpt and Gemini where I asked to explain me many theoretical things and asked for specific commands as well as help with errors.

CI/CD Pipeline

Reinis Janis Brencis

I owned Jenkins and the CI/CD side of the project — everything that happens between someone pushing code and the app actually being live on the server. By the end there was a fully automated pipeline running end to end with zero manual steps.

Jenkins setup

Jenkins was installed twice: first on a personal EC2 instance from an earlier lab, then a full reinstall on the shared instance once it became clear the whole project needed to run on one box, not split across two. Amazon Linux 2023 uses dnf instead of apt, which caused a couple of early mistakes, and this Jenkins version doesn't read the old config file for its port anymore — getting it onto 8081 instead of the default 8080 needed a systemd override instead. The jenkins user was also added to the docker group and given passwordless sudo, since Ansible needs to escalate privileges and nobody is around to type a password when Jenkins runs unattended.

The Jenkinsfile

Five stages: Checkout, Build Docker Image, Push to Docker Hub, Deploy to EC2, Health Check. Every build is tagged with the actual Jenkins build number instead of latest, so it's always clear exactly which version is running. The deploy stage simply calls the Ansible playbook — it used to be far more complicated before Ansible took over that responsibility.

GitHub webhook

Configured so a push to main triggers a build automatically, with no manual “Build Now” click. It took a few attempts to get fully working — the payload URL had to be fixed more than once as the instance's IP kept changing (resolved by attaching an Elastic IP), and anonymous read-only access eventually had to be enabled in Jenkins, since GitHub's webhook delivery is an unauthenticated request and Jenkins was silently rejecting it.

The SSH → SSM detour

This was the most difficult part. Once Accenture's security team required no more SSH across the whole cohort, the deploy stage was rebuilt around AWS SSM — a new IAM role, KMS permissions, all of it. Right after that finally worked, it turned out the whole project actually had to run on a single shared instance instead of two separate ones. That meant Jenkins moved onto the same box as the application, and the SSM work became unnecessary, since there was no longer a remote machine to reach into at all. In hindsight, a full day of debugging was made irrelevant by an architecture requirement that only surfaced afterward — not wasted time, though, since SSM is now genuinely well understood.

Bonus tasks

- GitHub webhook (automatic build triggering)
- Securely stored credentials in Jenkins — only one credential exists (Docker Hub), since AWS access goes through the instance's own IAM role instead of a stored key
- Versioned Docker images — build-number tags instead of latest; also caught and fixed that Ansible's side was still hardcoded to latest
- Wired blue/green deployment into the pipeline — Ansible performs the actual switch, Jenkins' job is triggering it correctly

What I'd improve with more time

- Scope the jenkins user's sudo access to specific commands instead of a blanket NOPASSWD
- Add a webhook secret for signature verification instead of leaving that field blank
- Ask about the “must run on one instance” requirement earlier — would have avoided the SSM detour entirely

End-to-End Architecture & Pipeline Overview

Everything described above runs together as a single system, on a single EC2 instance. This section walks through the full pipeline, start to finish, and how each person's part connects to the next.

The full flow

- 1. A developer pushes a code change to the main branch on GitHub.
- 2. GitHub's webhook fires a request to Jenkins, running on the same EC2 instance, port 8081.
- 3. Jenkins checks out the code, builds a new Docker image, and tags it with both the current build number and latest.
- 4. The image is pushed to Docker Hub (sandissarkovskis/devops-final-project), authenticated with a Docker Hub access token stored in Jenkins Credentials.
- 5. Jenkins triggers the Ansible playbook locally on the same instance, passing the new image tag as a variable.
- 6. Ansible pulls that image from Docker Hub and starts it as the Blue container on port 8082, then runs an HTTP health check.
- 7. Once Blue is confirmed healthy, Ansible starts the same image as the Green container on port 8083 and health-checks it too.
- 8. When Green passes its health check, Ansible regenerates the NGINX configuration to point at Green, validates it, and reloads NGINX — traffic switches with no downtime.
- 9. The old Blue container is only removed after the switch is confirmed successful, so the previous working version stays available the entire time the new one is being verified.
- 10. CloudWatch continuously receives CPU metrics and application logs from the instance in the background, independent of the deployment flow.

Where each piece runs

All of this happens on one EC2 instance (Amazon Linux 2023, t3.medium, provisioned by Terraform) — Jenkins, Ansible, NGINX, and both Docker containers live side by side on the same machine. This wasn't the original plan; Jenkins was initially going to run on a separate instance, and moving it onto the shared box partway through the project is what made SSH-based remote deployment unnecessary and simplified the whole pipeline to run everything locally.

Access and security

The instance only exposes two ports to the internet: 80 for NGINX (the public entry point to the application) and 8081 for Jenkins (its UI and the GitHub webhook endpoint). Ports 8082 and 8083, used internally for blue/green, are never exposed externally. SSH is fully disabled, both at the network level (no inbound rule for port 22) and at the OS level (the sshd service itself is stopped). All administrative access to the instance goes through AWS Systems Manager Session Manager, authenticated with each team member's own IAM identity and MFA — there is no shared key file.

The instance has a permanent Elastic IP attached, so it doesn't change if the instance restarts, which also keeps the GitHub webhook URL stable.

State and observability

Terraform's own state — the record of what infrastructure exists — is stored remotely in an S3 bucket with a DynamoDB table for locking, rather than on any one person's laptop. CloudWatch collects a CPU utilization alarm and an application log group, giving visibility into the instance's health independent of whether a deployment is currently running.

Why it's built this way

The design follows one central idea: nothing about deploying a new version should require a person to log into a server and run commands by hand. Terraform defines the infrastructure as code; Ansible defines the deployment procedure as code; Jenkins triggers that procedure automatically on every push; and the blue/green pattern means a bad deployment never takes down a working one, because the old version isn't removed until the new one has proven itself healthy.

AI Usage

Used Claude (Sonnet 5, Anthropic) mostly for troubleshooting and error fixes — walked through pretty much every Jenkins/pipeline bug as it came up (Docker permission errors, git branch conflicts, the SSM/KMS detour, heredoc and printf syntax bugs, sudo permission issues). Also relied on it for Amazon Linux 2023-specific syntax since it's dnf-based instead of apt and I hadn't worked with that before, plus help putting together my part of the documentation.